

ПАМЯТЬ И C++

ПАМЯТЬ И C++

1. Модель памяти в языке C
2. Модель памяти программы
3. Виртуальная память
4. Процессор и память
5. Move семантика

МОДЕЛЬ ПАМЯТИ В ЯЗЫКЕ C

Данные (переменные и функции) в программе можно охарактеризовать по:

1. Времени жизни
2. Области видимости
3. Типу связывания

Время жизни – продолжительность хранения объекта в памяти программы.

Область видимости – части исходного кода, из которых можно получить доступ к объекту по идентификатору.

Тип связывания – описывает отношение части исходного кода откуда можно обратиться к объекту по его имени, к той в которой определён объект.

ВРЕМЯ ЖИЗНИ

Время жизни:

1. Автоматическое (локальные переменные)
2. Статическое (статические и глобальные переменные)

Автоматическое – время жизни объектов равно времени выполнения охватывающего их блока.

Пример: локальные переменные без static.

Статическое – время жизни объектов равно времени выполнения программы.

Пример: глобальные переменные или локальные переменные со static.

```
1 // Статическое время жизни
2 static int var1;
3
4 // Статическое время жизни
5 int var2;
6
7 int main() {
8     // Автоматическое время жизни
9     int var3;
10
11     // Статическое время жизни
12     static int var4;
13 }
14
```



Автоматические переменные по умолчанию не инициализируются. Обращение к неинициализированным переменным вызывает UB.

Статические переменные по умолчанию инициализируются нулями. Инициализация статических переменных должна происходить константными выражениями.

ОБЛАСТЬ ВИДИМОСТИ

Область видимости:

1. Блок { }

(локальные и статические локальные переменные)

2. Функция

(локальные и статические локальные переменные)

3. Файл

3.1. Действует только в текущей единице трансляции
(глобальные статические переменные)

3.2. Действует во всех единицах трансляции, где
сделано объявление (глобальные переменные)



Любая функция автоматически видна во всех модулях программы. Если требуется ограничить область видимости (действия) функции файлом, в котором она описана, используется модификатор `static`.

```
1 // Область видимости var1 - функция.
2 // Область видимости func1 - все единицы трансляции, где подключена.
3 void func1(int var1);
4
5 // Область видимости func2 - текущая единица трансляции.
6 static void func2(int var2);
7
8 // Область видимости - все единицы трансляции, где подключена.
9 int var3;
10
11 // Область видимости - текущая единица трансляции.
12 static int var4;
13
14 int main() {
15     // Область видимости - блок {}.
16     int var5;
17
18     // Область видимости - блок {}.
19     static int var6;
20 }
21
```

ТИП СВЯЗЫВАНИЯ

Тип связывания:

1. Отсутствует
2. Внутренний
3. Внешний



Тип связывания играет важную роль при инициализации и использовании глобальных переменных.

Объекты с областью видимости блока и функции связывания не имеют.

Объект с внутренним типом связывания может использоваться только внутри файла в рамках той области видимости, где определён.

Объект с внешним типом связывания может использоваться в любом файле программы.

```
1  // Внешнее связывание
2  int var1;
3
4  // Внутреннее связывание
5  static int var2;
6
7  void func(int var3 /* Связывание отсутствует */);
8
9  int main() {
10     // Связывание отсутствует
11     int var4;
12
13     // Связывание отсутствует
14     static int var5;
15
16     /*
17      * Внешнее связывание.
18      *
19      * var6 - глобальная переменная, определённая в другой
20      * единице трансляции и, при этом, неподключенная через
21      * заголовочный файл в эту единицу трансляции.
22      */
23     extern int var6;
24 }
25
```

ПРИМЕР ВНЕШНЕГО СВЯЗЫВАНИЯ

Память под данные выделяется при её инициализации. `const` переменные надо инициализировать при объявлении. Если заголовочный файл содержит `const` переменные, то хочется избежать лишней работы компилятора и линкера: при каждом подключении заголовочного файла в единицу трансляции будут инициализироваться все `const` переменные. И так будет для каждой единицы трансляции. Чтобы этого избежать и инициализировать `const` переменные на помощь приходит механизм внешнего связывания и ключевое слово `extern`.

```
1  #pragma once
2
3  #include "menu.hpp"
4
5  namespace biv {
6      extern const MenuItem MAIN;
7
8      extern const MenuItem STUDY;
9      extern const MenuItem EXIT;
10
11     extern const MenuItem STUDY_SUMM;
12     extern const MenuItem STUDY_SUBSTRACT;
13     extern const MenuItem STUDY_MULTIPLY;
14     extern const MenuItem STUDY_DIVIDE;
15     extern const MenuItem STUDY_GO_BACK;
16 }
17
```

```
1  #include "menu_items.hpp"
2
3  #include <cstddef>
4
5  #include "menu_functions.hpp"
6
7  const biv::MenuItem biv::STUDY_SUMM = {
8      "1 - Хочу научиться складывать!", biv::study_summ, &biv::STUDY
9  };
10 const biv::MenuItem biv::STUDY_SUBSTRACT = {
11     "2 - Хочу научиться вычитать!", biv::study_substract, &biv::STUDY
12 };
13 const biv::MenuItem biv::STUDY_MULTIPLY = {
14     "3 - Хочу научиться умножать!", biv::study_multiply, &biv::STUDY
15 };
16 const biv::MenuItem biv::STUDY_DIVIDE = {
17     "4 - Хочу научиться делить!", biv::study_divide, &biv::STUDY
18 };
19 const biv::MenuItem biv::STUDY_GO_BACK = {
20     "0 - Выйти в главное меню", biv::study_go_back, &biv::STUDY
21 };
22
```

`const` заставил глобальную переменную инициализироваться в месте объявления.

`extern` исправил эту ситуацию и «вернул» все возможности внешнего связывания.

КЛАССЫ ПАМЯТИ ЯЗЫКА C

Классы памяти языка C:

1. Автоматический
2. Регистровый
3. Статический без связывания
4. Статический с внутренним связыванием
5. Статический с внешним связыванием

КЛАССЫ ПАМЯТИ ЯЗЫКА C

КЛАСС ПАМЯТИ	ВРЕМЯ ЖИЗНИ	ОБЛАСТЬ ВИДИМОСТИ	ТИП СВЯЗЫВАНИЯ	МЕСТО ИНИЦИАЛИЗАЦИИ
Автоматический	Автоматическое	Блок	Отсутствует	Блок
Регистровый	Автоматическое	Блок	Отсутствует	Блок
Статический без связывания	Статическое	Файл	Отсутствует	Блок
Статический с внутренним связыванием	Статическое	Файл	Внутренний	Вне функций
Статический с внешним связыванием	Статическое	Файл	Внешний	Вне функций и не static

ИЗМЕНЕНИЯ В C++

Изменения в C++ модели памяти языка:

1. Регистровый класс памяти
2. Область видимости:
 - + Класс
 - + Именованная область

МОДЕЛЬ ПАМЯТИ ПРОГРАММЫ

Когда ОС запускает программу, то она создаёт сущность «процесс», привязывает к нему адресное пространство оперативной памяти и в нём держит все ресурсы программы. Далее, внутри процесса, ОС создаёт сущность «поток» и в нём запускает функцию `main`.

Процесс — это экземпляр выполняющейся программы, который имеет свое собственное виртуальное адресное пространство и ресурсы, такие как файловые дескрипторы и переменные окружения. Каждый процесс имеет как минимум один поток выполнения, который выполняет инструкции программы.

Поток — это отдельная единица выполнения в рамках процесса. Каждый поток имеет свой собственный стек и регистры процессора, но разделяет общее адресное пространство и ресурсы с другими потоками в рамках одного процесса.

МОДЕЛЬ ПАМЯТИ ПРОГРАММЫ



В верхней части адресного пространства находится стек.

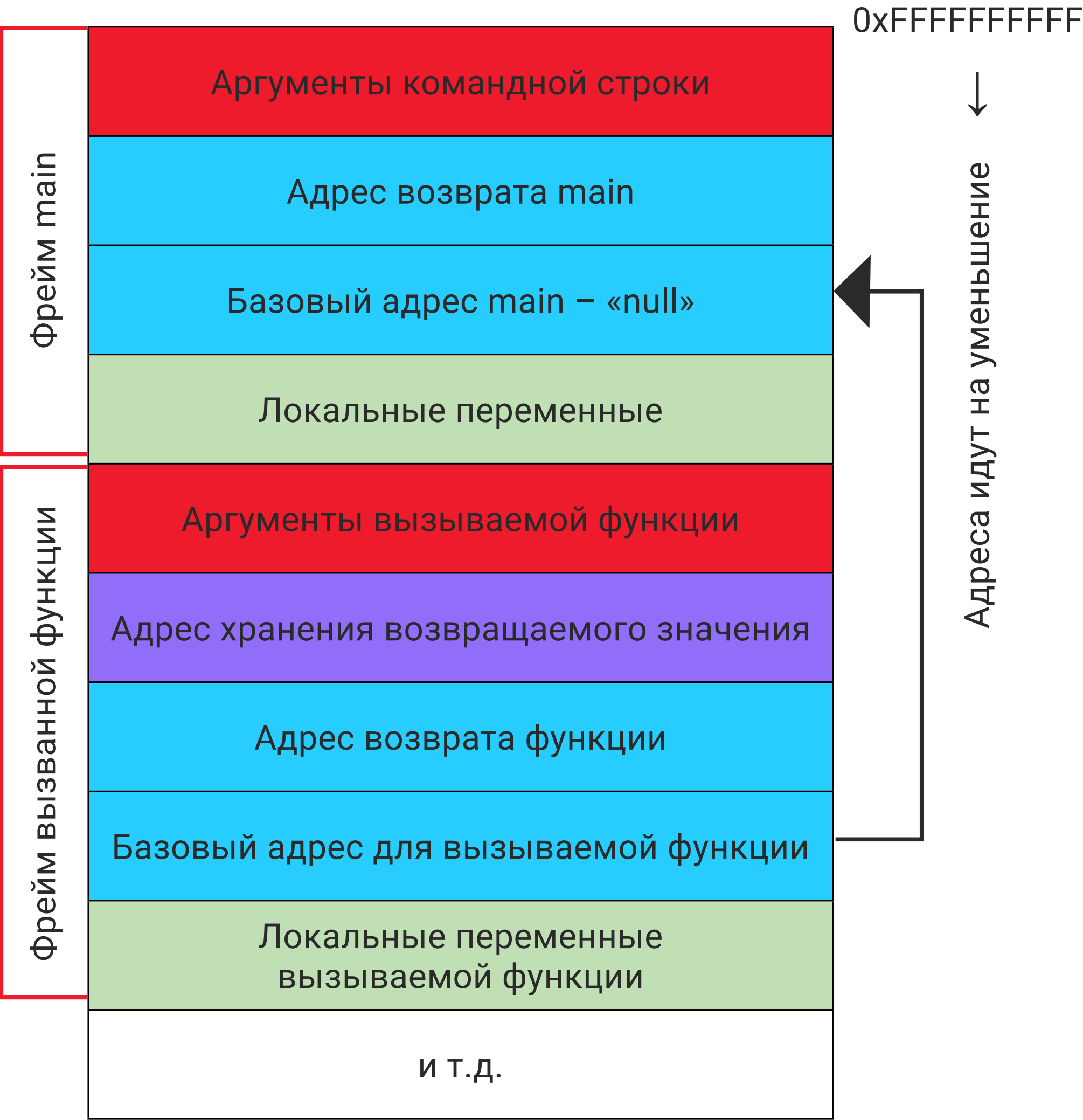
Стек (Stack):

- 1. Относится к потоку, а не к процессу.
- 2. Его размер ограничен и равен, как правило, 1 Мб.
- 3. Адреса хранящихся данных идут на уменьшение.
- 4. Хранит переменные автоматического класса памяти, размер которых можно вычислить во время компиляции.
- 5. Данные обрабатываются по принципу LIFO «Последний пришёл – первый вышел».
- 6. Данные расположены последовательно.

Куча (Heap):

- 1. Относится к процессу.
- 2. Размер не ограничен.
- 3. Адреса идут на возрастание, на встречу стеку.
- 4. Хранит переменные автоматического класса памяти, размер которых нельзя вычислить во время компиляции.
- 5. Данные расположены произвольно.

Утечки памяти (memory leak) – неконтролируемое уменьшение оперативной памяти.



Стек создаётся, когда запускается поток для функции `main()` – главный поток программы. В многопоточных программах стек создаётся при запуске каждого потока.

Стек состоит из фреймов функций, которые содержат:

1. Аргументы функции.
2. Если функция возвращает значение, то добавляется память под возвращаемое значение.
3. Адрес возврата функции.
4. Базовый адрес фрейма.
5. Локальные переменные.

Фрейм функции (кадр стека) – это часть стека, предназначенная для хранения всех необходимых данных для выполнения функции.

Адрес возврата - это адрес инструкции, следующей за командой вызова функции.

Базовый адрес фрейма указывает на «начало» фрейма, из которого была вызвана функция.

Фрейм main	Аргументы main
	Адрес возврата main
	«null»
	result
Фрейм summ	y = 10
	x = 5
	&result
	main.cpp:8
	main(0)
	summ

СТЕК

Пример расположения данных программы в стеке

```
1 int summ(int x, int y) {
2     int summ = x + y;
3     return summ;
4 }
5
6 int main() {
7     int result;
8     result = summ(5, 10);
9 }
10
```



Приведите пример переполнения стека. На основании структуры стека объясните почему это произойдёт.

Куча хранит переменные автоматического класса памяти, размер которых на момент компиляции не известен. Т.е. память под данные этих переменных выделяется во время выполнения программы и зависит от значений, которые приходят в программу. Программист должен заботиться о выделении и освобождении памяти.

Выделение и освобождение памяти в C

1. Функции `malloc` и `calloc` выделяют память.

malloc выделяет для нужд процесса непрерывный фрагмент оперативной памяти, складывающийся из блока запрошенного объёма (адрес, которого возвращается как результат функции) и следующего перед ним блока служебной информации, имеющей длину в несколько байт и содержащего размер выделенного блока и ссылку на следующий блок памяти в связанном списке блоков.

Пример: `int* mheap = malloc(sizeof(int));`

Замечание: выделенный блок памяти является универсальным указателем `void*` и приводится к нужному типу. Но он остаётся неинициализированным.

Внутри `malloc` есть буфер. Если в буфере есть место, то вызов выделения памяти выполнится быстро.

Выделение и освобождение памяти в C++

1. Оператор `new` и `new[]` выделяют память:

new делает то же самое, что и `malloc`. Но для одного объекта указанного типа. И инициализирует объект вызовом конструктора.

new[] выделяет память под `n` элементов указанного тип и инициализирует их, вызывая конструкторы по умолчанию.

При неудачном выделении памяти `new` выбрасывает исключение `std::bad_alloc`.

Выделение и освобождение памяти в C

Функция `calloc` выделяет память для `n` элементов по `m` байт. В отличие от `malloc` она инициализирует все элементы нулями.

Пример: `int *cheap = calloc(2, sizeof(int));`

При неудачном выделении памяти обе функции возвращают нулевой указатель (`NULL`).

2. **`realloc()`** позволяет изменить размер памяти, ранее выделенной с помощью функций `malloc()` и `calloc()`.

`void* realloc(void* ptr, unsigned size);`

`realloc` также ничего не знает о конструкторах.

3. **`free`** помечает ранее выделенный фрагмент памяти как свободный.

Если `free` передать неверный указатель, то будет UB.

Двойное освобождение памяти тоже приведёт к UB.

Выделение и освобождение памяти в C++

2. `malloc`, `calloc`, `realloc`, `free` перегрузить нельзя, а операторы `new`, `new[]`, `delete`, `delete[]` перегрузить можно.

3. **`delete`** и **`delete[]`**

`delete` вызывает деструктор объекта соответствующего типа.

`delete[]` вызывает деструкторы всех объектов массива.

Выделение и освобождение памяти в C

Выделение и освобождение памяти в C++

При повторном применении `delete` и `delete[]` к соответствующему указателю будет UB.

4. Если вместо `delete[]` применить `delete`, то будет UB.

`delete[]` и `delete` – это два разных оператора с разными алгоритмами освобождения памяти и разной служебной информацией относительно выделенной памяти по указанному указателю.

5. **placement new** в C++ или «размещение»

В C++ можно проинициализировать объект по уже выделенной памяти, т.е. тем самым избежать лишних действий по освобождению и выделению памяти. А поместить новый объект на место уже ненужного.

```
int a;
```

```
int* ptr = new(&a) int;
```

КУЧА



Запомнить!

1. Если выделенную память не освободить, то это приведёт к утечке памяти (memory leak). Неосвобождённая память в итоге освободится только при завершении программы, т.к. ОС убьёт процесс, созданный под программу. И, соответственно, освободит все связанные ресурсы.
2. Освобождение памяти не приводит к занулению указателя. Из-за этого в C++ часто применяют следующую технику:

```
delete ptr;  
ptr = nullptr;
```
3. Нельзя мешать буратино с крем-содой, т.е. заниматься выделением и освобождением памяти функциями языка C в C++. В C++ используйте всегда нужные формы new и delete.

ВИРТУАЛЬНАЯ ПАМЯТЬ

Проблемы использования настоящей памяти

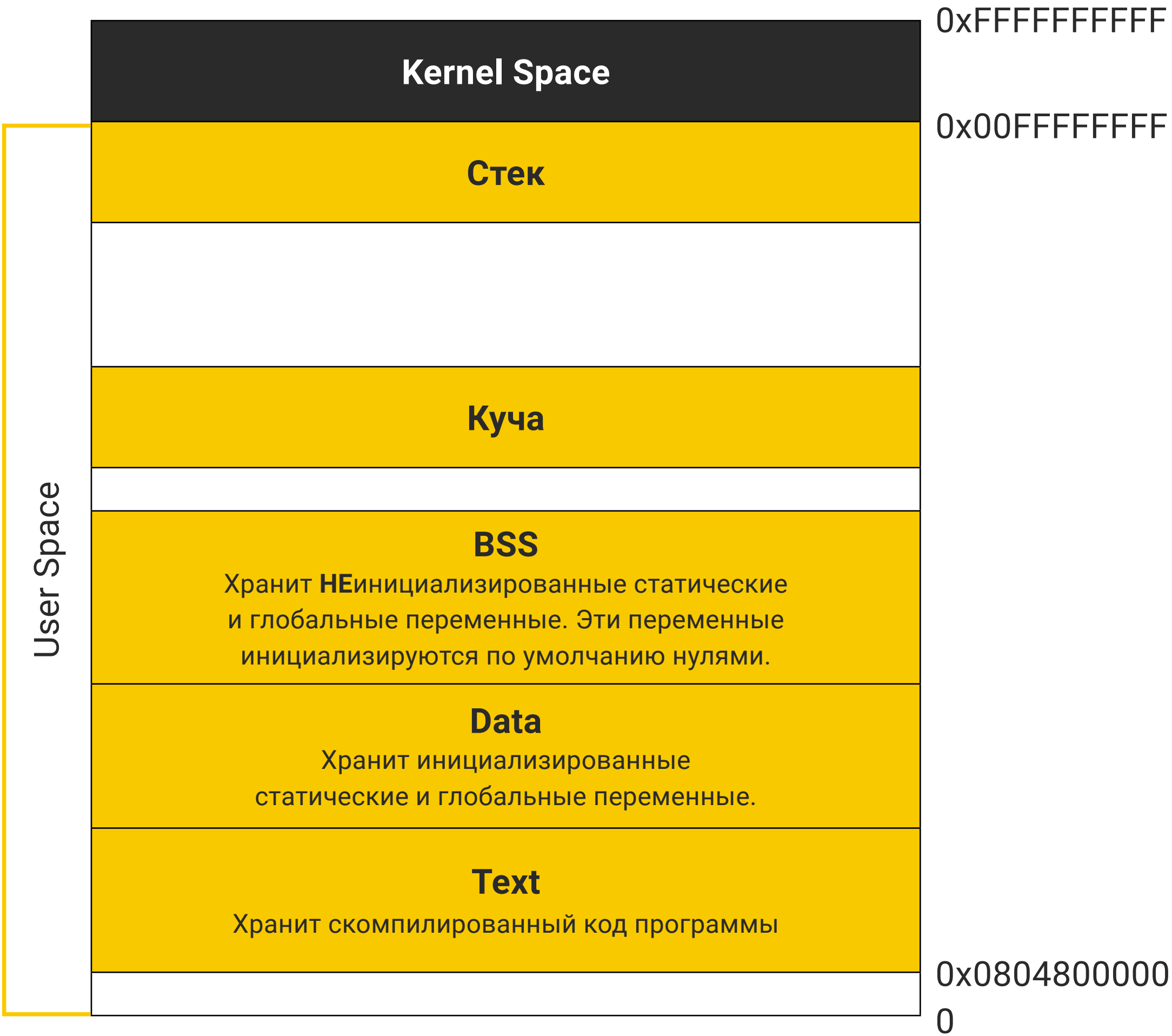
1. Программе требуется больше памяти, чем есть оперативной памяти.
2. Надо согласовать использование общей памяти между программами.
3. Изолировать программы друг от друга путём назначения непересекающихся адресов.

Или другими словами:

1. Во время работы программы может возникнуть нехватка ОЗУ.
2. Программист не должен отвечать за физическую организацию памяти.
3. Программист не должен знать, где в памяти будет находиться программа.

МОДЕЛЬ ПАМЯТИ ПРОГРАММЫ

Модель вирутальной памяти программы



Каждый процесс (программа) в ОС работает в виртуальном, а не настоящем, адресном пространстве.

В 32-битном режиме оно имеет размер 4 Гб, в 64-битном – соответственно больше.

Соответствие между виртуальным пространством и физической памятью описывается с помощью таблицы страниц. ОС создает и заполняет таблицы, а процессор через MMU (memory management unit) обращается к ним при необходимости осуществить трансляцию адреса.

Для обеспечения взаимодействия прогаммы и ОС часть виртуальной памяти процесса всегда зарезервированно под работу с ОС. Эта часть называется kernel space, а та, в которой находится уже известная нам модель памяти программы – user space.

Каждый процесс имеет в своём распоряжении полный диапазон адресов виртуальной памяти, кроме kernel space, и свою таблицу страниц соответствия физическим адресам.

ДВА КИТА

Механизм виртуальной памяти обеспечивается таким компонентом процессора как MMU и таким модулем ОС как менеджер виртуальной памяти.

MMU (memory management unit – блок управления памятью) – компонент аппаратного обеспечения компьютера (часто входит в состав центрального процессора), отвечающий за управление доступом к памяти, запрашиваемым центральным процессором. Его функции заключаются в трансляции адресов виртуальной памяти в адреса физической памяти, защите памяти, управлении кэш-памятью, шиной и, в более простых компьютерных архитектурах, переключением блоков памяти.

Менеджер виртуальной памяти – часть ОС, благодаря которой можно адресовать память большую, чем объем физической памяти.

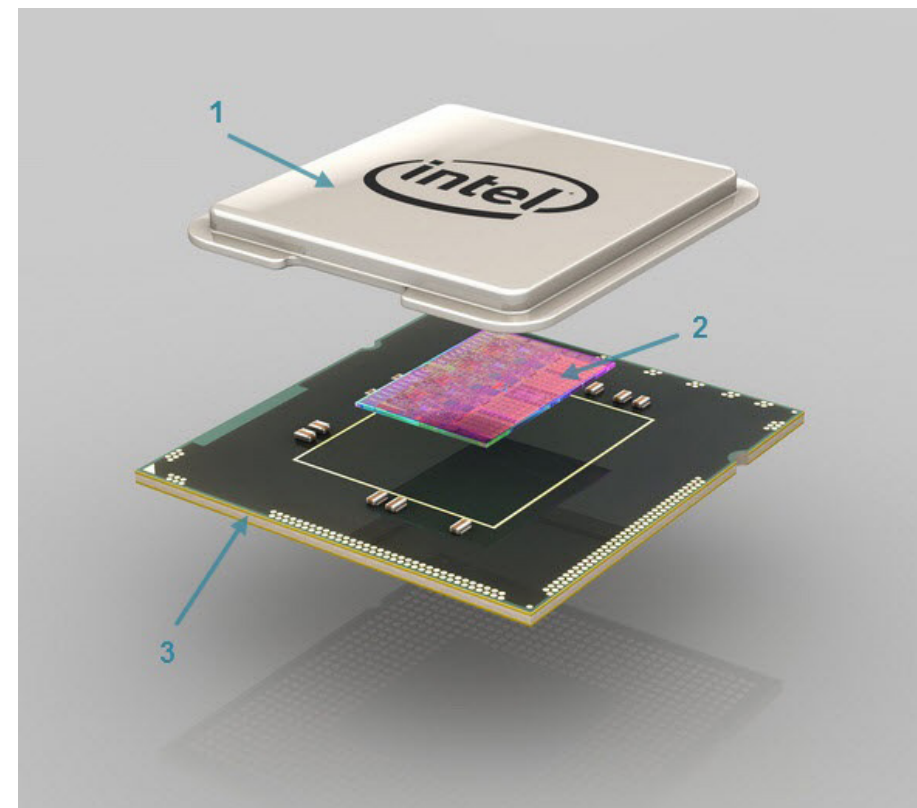
Процессор

- Интегральная схема
- АЛУ
- Регистры
- Управляющий блок
- Тактовый генератор
- Архитектура
- Архитектура фон Неймана
- Гарвардская архитектура
- Параллельная архитектура
- CISC / RISC / MISC / VLIW
- Такт процессора / Тактовая частота
- Производительность ЭВМ / Флопс
- x86
- Передача управления

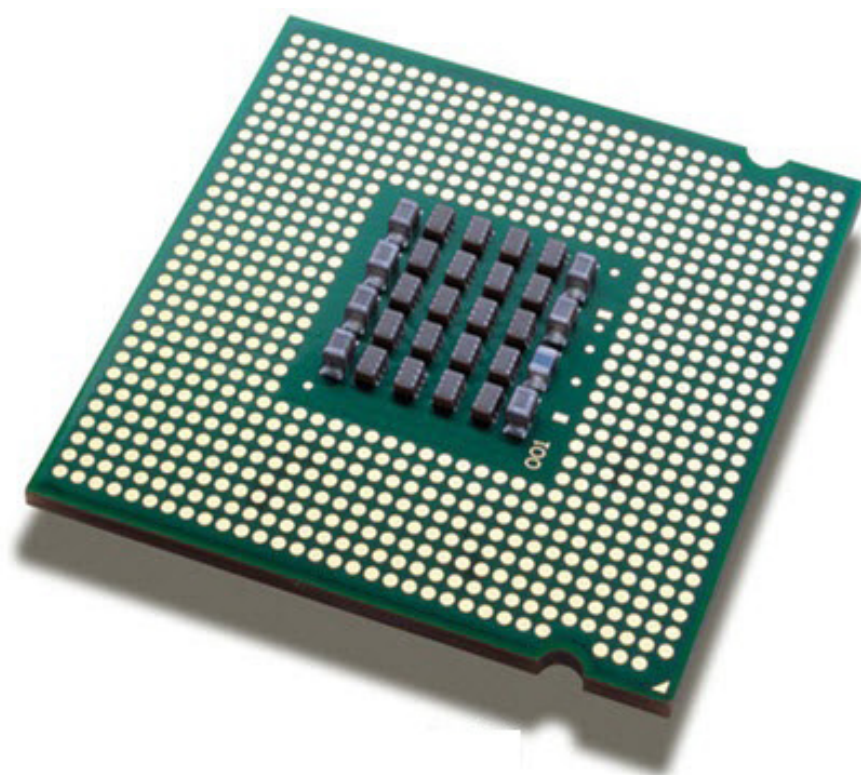
Память

- Разрядность процессора / машинное слово
- Шина данных / шина адреса
- ОЗУ (RAM) / ПЗУ (ROM)
- Иерархия памяти / время доступа к данным
- Регистры процессора
- Кэш (1-го уровня, 2-го уровня, 3-го уровня)
- RAM (ОЗУ / Виртуальная память)
- Постоянные хранилища (ПЗУ / ROM BIOS-а, жёсткий диск)
- Память устройств ввода / вывода, периферийных устройств
- Кэш промах

ПРОЦЕССОР



1. Металлическая крышка
2. ИС
3. Контактная подложка



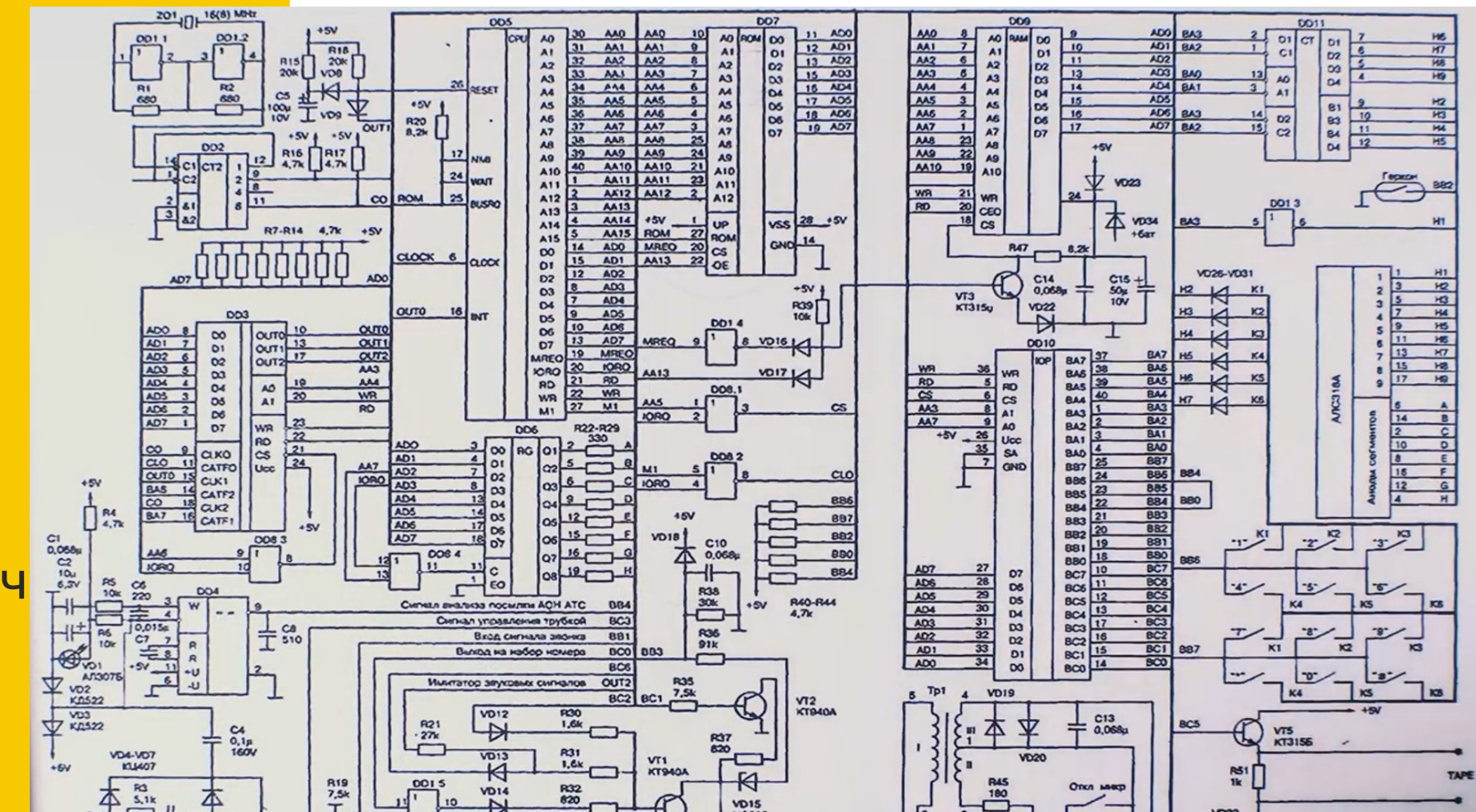
Контактная подложка. Перевернутый вид

Процессор – это интегральная схема (ИС, или микросхема, или кристалл, или чип), которая выполняет машинные инструкции.

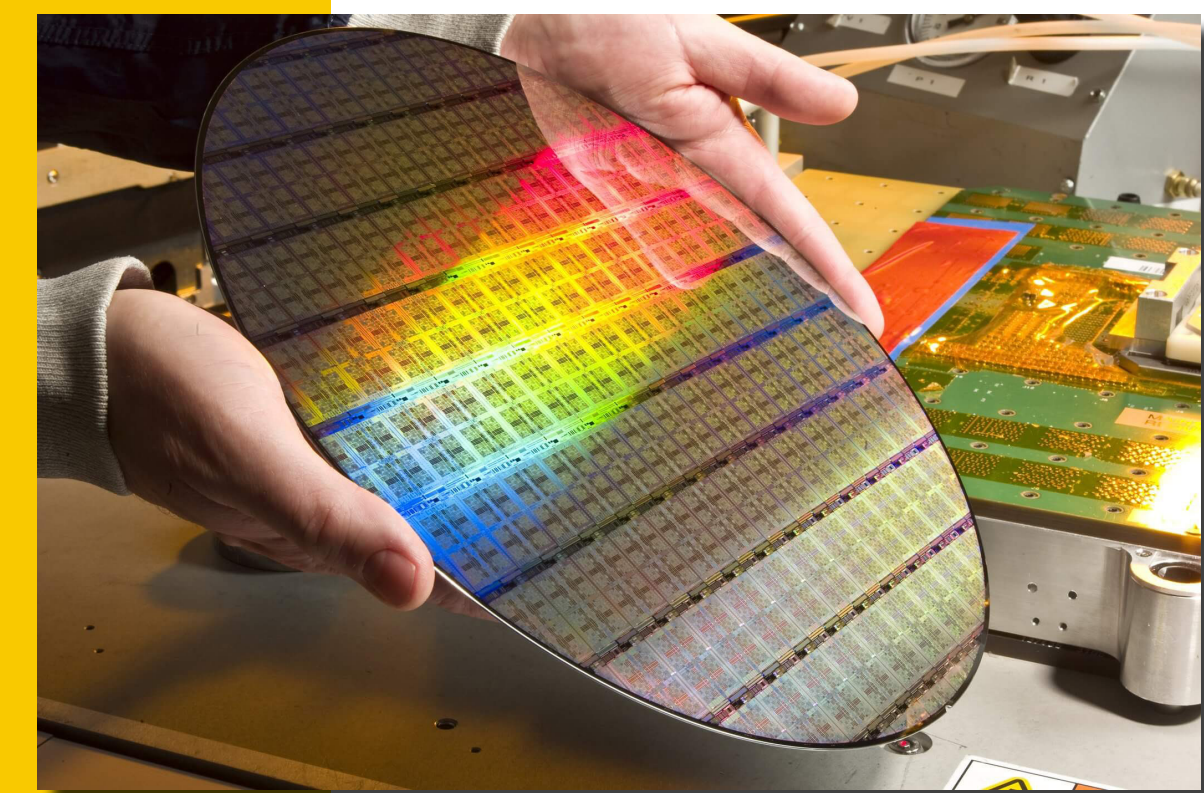
Интегральная схема – это электронная схема на полупроводниковой подложке. На кремниевой пластине размещены сотни миллионов транзисторов-переключателей и каналов передачи данных.

ИС находится на контактной подложке, через которую соединяется с материнской платой. А сверху закрывается металлической крышкой, которая выполняет функцию защиты и теплоотделения.

О том как производятся процессоры можно посмотреть в видео:
<https://www.youtube.com/watch?v=z9r-I9PdIac>

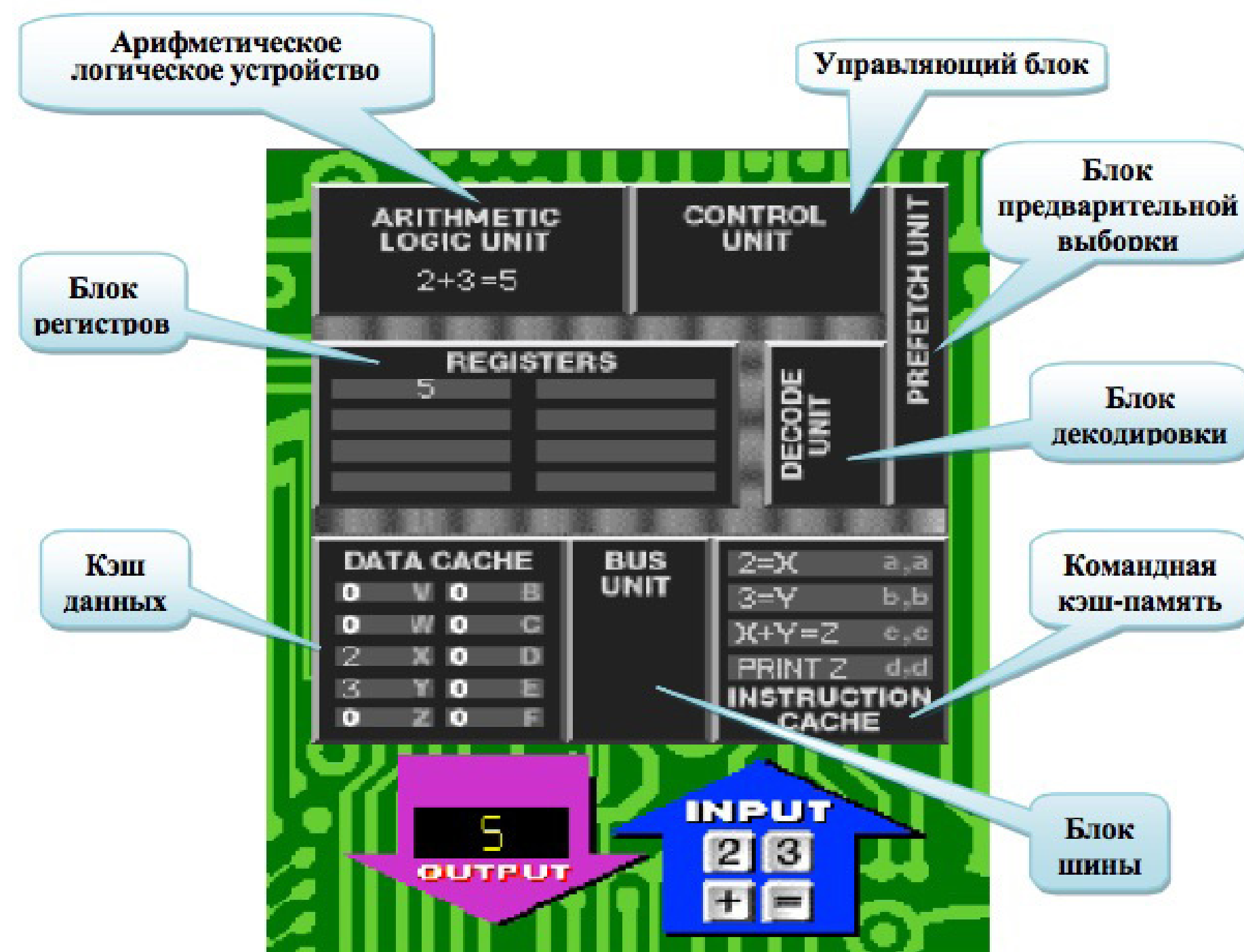


Интегральная схема



Пластина кристаллов на стадии производства

Основные компоненты процессора



Основные компоненты процессора:

АЛУ (Арифметическое логическое устройство) – это устройство, которое выполняет готовые к исполнению команды и заносит результаты в блок регистров.

Регистры – это поля заданной длины внутри процессора сверхбыстрой оперативной памяти. Они нужны для хранения данных о работе процессора и хранения инструкций и данных, участвующих в вычислениях. Регистры бывают специального и общего назначения.

Управляющий блок (Control Unit) – это блок, который занимается согласованием работы частей самого процессора и его связи с другими устройствами.

Кэш – это память, которая хранит часто используемые данные и команды, что значительно ускоряет к ним доступ.

Тактовый генератор – это устройство, которое задаёт рабочую частоту процессора. С помощью тактовых импульсов выполняется синхронизация для внутренних команд процессора и остальных устройств.

Архитектура (с аппаратной точки зрения) – это набор свойств и качеств, присущий семейству процессоров.

Основные виды архитектур:

1. Архитектура фон Неймана (x86)

2. Гарвардская архитектура

3. Параллельная архитектура (суперкомпьютеры)

1. **Архитектура фон Неймана** – это принцип совместного хранения команд и данных в памяти компьютера.

Узким местом является пропускная способность между процессором и памятью, т.к. данные и инструкции должны идти последовательно.

2. **Гарвардская архитектура** – это принцип хранения данных и инструкций на разных аппаратных устройствах. Работа с данными и инструкциями осуществляется параллельно.

3. **Параллельная архитектура** – это возможность одновременно обрабатывать массивы данных. Архитектуры фон Неймана и Гарвардская являются последовательными.

Архитектура (с программной точки зрения) – это набор команд (инструкций), которые умеет выполнять процессор, и набор регистров, описывающие особенности памяти процессора.

С программной точки зрения архитектуры бывают:

1. CISC (Complex Instruction Set Computer – «компьютер с полным набором команд»). Пример – x86

2. RISC (Reduced Instruction Set Computer – «компьютер с сокращённым набором команд»). Пример – ARM

3. MISC (Minimal Instruction Set Computer – «компьютер с минимальным набором команд»). Пример – IoT.

4. VLIW (Very Long Instruction Word – «очень длинная машинная команда»). Пример – суперкомпьютеры.

Производительность

Такт процессора — это промежуток времени между двумя импульсами тактового генератора, который синхронизирует выполнение всех операций процессора.

Тактовая частота — количество тактов, поступающих извне на вход схемы за одну секунду.

Производительность ЭВМ — это количественная характеристика скорости выполнения определённых операций на компьютере. Чаще всего производительность ЭВМ измеряется во флопсах.

Флопс — это количество операций с плавающей запятой в секунду.

Оценка реальной вычислительной мощности производится путём прохождения специальных тестов (бенчмарков). Обычно оценивается скорость решения большой системы линейных алгебраических уравнений. Наиболее популярным тестом производительности является Linpack benchmark.

Разрядность и память

Разрядность процессора (машинное слово) — это величина, которая определяет сколько единиц информации (битов) процессор может обработать (прочитать или записать) за 1 такт.

Также эта величина определяет и размер регистров.

А ещё эта величина говорит о максимально возможной величине адреса виртуальной памяти (т.к. именно эти адреса могут храниться в регистрах). И, соответственно, ограничении на размер ОЗУ. Например, в 32-разрядных машинах максимальный размер ОЗУ равен 4 Гб. Ставить больше ОЗУ бессмысленно.

Процессор общается с ОЗУ (RAM) и ПЗУ (ROM) с помощью шины данных, шины адреса и шины управления. Разрядность шины данных равна разрядности процессора. А разрядность шины адреса зависит от размера всей адресуемой памяти в компьютере.

Самой распространённой программной архитектурой является x86. Она была разработана компанией Intel в 1978 г. и стала применяться большинством производителей компьютеров: настольных, ноутбуков и даже суперкомпьютеров.

Эта архитектура является CISC архитектурой, т.е. для многих арифметических действий имеется специально выделенная команда.

Контрпримером являются мобильные устройства, которые в большинстве случаев используют ARM архитектуру. ARM является RISC архитектурой. Её применение связано с ограниченностью мобильных устройств в аппаратном плане: размеры, мощность, энергопотребление и т.п.

Большинство всех ЭВМ аппаратно используют архитектуру фон Неймана. Принципы Гарвардской архитектуры могут быть реализованы для какой-то отдельной части ЭВМ, не исключая при этом общего построения по принципам фон Неймана.

Иерархия памяти ЭВМ



Место памяти в приведённой иерархии означает её удалённость от процессора и, соответственно, время, требуемое для чтения / записи данных. Далее приведено время доступа к данным из процессора в зависимости от места в иерархии:

1. Регистры процессора – 1-2 такта
2. Кэш
 - 1-го уровня – 2-3 такта
 - 2-го уровня – 3-5 тактов
 - 3-го уровня – 30-50 тактов
3. Оперативная память – 50-200 тактов
4. Жёсткий диск – 50 000 тактов



При написании программ, работающих с большими массивами данных важно понимать устройство памяти, скорость доступа к разным видам памяти, расположение ваших данных в этих видах памяти во время работы программы.

ПРОЦЕССОР И ПАМЯТЬ



Скорость работы программы зависит не только от количества операций в алгоритмах, но и очень сильно от суммарного времени, потраченного процессором на чтение / запись данных. А это требует хорошего понимания расположения ваших данных в памяти во время работы программы.

Передать управление – это передать процессору адрес, по которому находится первая инструкция другой программы.

1. Загрузка BIOS в ОЗУ

После включения компьютера процессор начинает читать код расположенный с 0-го адреса из загрузать его в ОЗУ. По 0-му адресу в ЭВМ находится ПЗУ ROM, в котором находится BIOS (Basic input / output system) – базовая система ввода / вывода.

2. Начальная настройка ЭВМ BIOS-ом

Когда BIOS загружен в ОЗУ, то он начинает выполнять начальную загрузку компьютера, передавая процессору набор своих инструкций.

3. Передача BIOS-ом процессору адреса расположения ОС

Начальная загрузка заканчивается, когда BIOS передаёт процессору адрес, с которого начинается ОС.

4. Загрузка ОС в ОЗУ

После загрузки ОС в ОЗУ BIOS передаёт управление ОС. Процессор начинает исполнять инструкции ОС.

5. Настройка ЭВМ ОС-ой для полноценной работы

ОС – это обычная программа, но только большая, сложная и её задачей является настройка взаимодействия всего аппаратного обеспечения ЭВМ.

6. Готов к труду и обороне!

После окончания настройки ЭВМ процессор занимается отслеживанием её состояния (и самого себя заодно: «А чем я хуже?»). Он постоянно читает регистры, отвечающие за события. Эти события называются прерываниями. Когда происходит какое-то событие, то процессору в определённый регистр записывается вид прерывания.

7. Запуск программы пользователем. Создание процесса и модели виртуальной памяти

Когда пользователь запускает программу, то генерируется программное прерывание. И процессор начинает исполнять соответствующие ему инструкции ОС. ОС создаёт процесс, «привязывает» к нему область ОЗУ и загружает туда программу. После этого ОС создаёт модель виртуальной памяти.

8. Подготовка к запуску программы

Текст программы загружается в область Text. Далее инициализируются статические данные Data и BSS, определяется область для кучи и запускается сам код программы.

9. Создание главного потока и стека

Но прежде запуска кода программы ОС в процессе создаёт главный поток. К потоку «привязывает» область памяти в модели, которая называется стеком.

10. Начало выполнения программы

И только после создания стека программе передаётся управление и начинается её непосредственное выполнение. Т.е. процессору начинают поступать инструкции программы.

ЗАЯЦ И ЧЕРЕПАХА

Или процессор и кэш

Во время выполнения программы её данные хранятся в ОЗУ. Доступ к данным (читать и писать) в ОЗУ у процессора занимает 50-200 тактов, а в кэше намного меньше. Поэтому по возможности процессор будет копировать данные из ОЗУ в кэш. Особенно, Если имеется работа с массивами данных. Алгоритм чтения данных следующий:

0. В кэше ничего нет.

1. Процессор читает первый элемент массива из ОЗУ и работает с ним согласно алгоритму программы.

2. Кэш контроллер (элемент процессора вместе с кэш памятью) копирует последовательный кусок данных, начинающийся с адреса прочитанного элемента. И распределяет его по кэшам.

3. Процессор согласно алгоритму запрашивает какой-то другой элемент массива.

4. Кэш контроллер последовательно от 1-го к 3-му уровню кэшей проверяет наличие запрашиваемого элемента. Если он там уже есть, то отдаёт процессору. Если нет, то процессор отправляется за элементом в ОЗУ, а кэш-

контроллер заново читает кусок памяти из ОЗУ, начинающийся уже с этого запрашиваемого элемента. И распределяет его по кэшам, затирая там старые данные.

Кэш промах – это ситуация, когда запрашиваемых данных не оказалось ни в одном из кэшей и чтение данных приходится делать из ОЗУ.

Оптимизация чтения данных для выполнения алгоритма состоит в том, чтобы написать алгоритм так, чтобы кэш промахов было наименьшее количество. Или другими словами, чтобы процессор как можно больше работал с данными, которые идут последовательно друг за другом.

ЗАДАЧИ НА КЭШ-ПРОМАХИ

№ 1. Суммирование элементов матрицы

Написать две программы, суммирующие элементы матрицы целых чисел. Одна программа суммирует по строкам, а вторая – столбцам. Измерить время работы и сравнить результаты. Тесты провести для матриц: 1000×1000, 10 000×10 000, 20 000×20 000 и т.д. пока компьютер в состоянии выделять память и считать результат. Теоретически посчитать разницу в количестве кэш-промахов согласно кэш памяти вашего компьютера.

№ 2. Произведение матриц

Классическим примером задачи, требующей неэффективного, с точки зрения архитектуры ЭВМ, обхода матрицы по столбцам, является задача об умножении матриц: $(AB)_{ij} = \sum_{k=0}^{n-1} a_{ik} \cdot b_{kj}$, – имеющая следующее очевидное решение:

Написать правильное с точки зрения ЭВМ умножение матриц. Сделать аналогичные задаче № 1 измерения времени, сравнения, тесты и теоретические расчёты кэш-промахов.

```
1 // Размеры матриц a[l][m], b[m][n], c[l][n]
2 for (int i = 0; i < l; i++) {
3     for (int j = 0; j < n; j++) {
4         int cij = 0;
5         for (int k = 0; k < m; k++) {
6             cij += a[i][k] * b[k][j];
7         }
8         c[i][j] = cij;
9     }
10 }
11
```

УМНОЖЕНИЕ МАТРИЦ

Правильное решение с точки зрения ЭВМ: чтения / записи данных в память, т.е. минимизации кэш-промахов, имеет вид:

```
1 // Размеры матриц a[l][m], b[m][n], c[l][n]
2 for (int i = 0; i < l; i++) {
3     for (int k = 0; k < m; k++) {
4         int aik = a[i][k];
5         for (int j = 0; j < n; j++) {
6             c[i][j] += aik * b[k][j];
7         }
8     }
9 }
10
```

MOVE СЕМАНТИКА

Переменная – это именованная область памяти, в которой хранятся данные определённого типа.

Указатель – это переменная, которая содержит адрес, по которому находятся данные.

Ссылка – это псевдоним (ещё одно / вымышленное имя) данных, через которое можно работать с данными.

В языке C были только переменные и указатели, а ссылки появились в C++. Ссылки по сравнению с указателями имеют ряд ограничений, а по сравнению с переменными не требуют дополнительной памяти при передаче данных в функцию или возврате из неё.

MOVE СЕМАНТИКА



Скорость работы программы зависит не только от количества операций в алгоритмах и суммарного времени, потраченного процессором на чтение / запись данных, но и от количества копирований данных (и соответственно, выделения и освобождения памяти).

Описание проблемы

При передаче данных в функцию или возврате из нее через переменную (по значению) происходит создание копии данных, которые та переменная именovala.

Создание копии данных – это выделение новой памяти, копировании в эту память данных, а по выходе из области видимости соответствующей переменной освобождение памяти.

*** Оптимизации компилятора, которые он может делать, не рассматриваются.

Для данных примитивных типов – эти операции происходят быстро, т.к. данные примитивных типов занимают $\leq$ одного машинного слова.

Для пользовательских типов, которые в себе содержат много полей и могут суммарно занимать много места, эти операции становятся дорогими.

Решение

Передача / получение объекта пользовательского типа через указатель или по ссылке частично решает описанную проблему. Но это не всегда является правильным решением по другим причинам или может быть невозможным. В C++11 версии был разработан механизм, который получил название Move семантика.

Move семантика (семантика перемещения) – это механизм, который позволяет данным быть «перемещенными» из одного объекта в другой, вместо создания их копии.

Цель move семантики – это оптимизировать вычислительные расходы там, где до этого применялась передача данных по значению (через переменную).

ПРИМЕР

```
1 String get_text() {  
2     String text = "Текст";  
3     // code...  
4     return text;  
5 }  
6  
7 int main() {  
8     String str = get_text();  
9 }  
10
```

В приведённом примере функция `get_text()` возвращает объект класса `String` по значению. Рассмотрим сколько раз в данном примере выделяется / освобождается память и копируются данные.

Строка 2 создаёт локальный объект `text`: выделяет память и записывает в неё данные «Текст».

Строка 4 возвращает объект `text` из функции в `main` в строку 8, где создаётся объект `str`: выделяется память и копируются данные «Текст».

После выполнения строки 8 выполняется строка 5 «завершение области видимости» функции `get_text()`. В этой строке вызывается деструктор класса `String`, и освобождается память из-под переменной `text`.

Вопрос: можно ли не выполнять каких-то действий, чтобы тем самым ускорить работу программы?

Ответ: да.

Т.к. после строки 8 на данные «Текст» будет ссылаться только одна переменная `str`, и ситуаций, при которых с этими данными может произойти UB, возникнуть не может, то в строке 8 можно скопировать ссылку на «Текст» из объекта `text` в объект `str`. Тогда не нужно будет делать:

1. Выделения памяти в строке 8 под `str`.
2. Копирования данных в строке 8.
3. Освобождения памяти в строке 5.

И подобных ситуаций в программах много.

Для реализации move семантики в C++ введены:

1. Понятие временного объекта или rvalue ссылки.
2. Конструктор перемещения.
3. Оператор присвоения перемещения.
4. `std::move()`

Временный объект – это объект, у которого отсутствует необходимость сохранять свои внутренние ресурсы для дальнейшего использования.

rvalue ссылка – это ссылки, представляющие временные объекты.

rvalue ссылки обозначают как T&&.

`std::move()` преобразует lvalue в rvalue, позволяя использовать конструктор перемещения или оператор присвоения перемещения.

```
1 ~ String get_text() {  
2     String text = "Текст";  
3     // code...  
4     return text;  
5 }  
6  
7 ~ int main() {  
8     String str = get_text();  
9 }  
10
```

В рассмотренном примере объект «Текст» при возврате из функции получает статус временного объекта, т.к. после выполнения строки 8 данные, которые относятся к этому объекту, сразу же будут удалены из памяти.

Поэтому можно не копировать данные, а «переместить» их в `str`. Указателям же временного объекта просто присваивается `nullptr` без освобождения памяти. Для реализации «перемещения» в строке 8 будет вызван конструктор перемещения класса `String`.

РЕАЛИЗАЦИЯ КЛАССА STRING

```
1 class String {
2     private:
3         char* data;
4         std::size_t size;
5     public:
6         String(const char * const str) {
7             size = std::strlen(str) + 1;
8             data = new char[size];
9             std::strcpy(data, str);
10        }
11        String(const String& other) {
12            size = other.size;
13            data = new char[size];
14            std::strcpy(data, other.data);
15        }
16        String(String&& other) {
17            size = other.size;
18            data = other.data;
19            other.data = nullptr;
20            other.size = 0;
21        }
22        String& operator = (const String& other) {
23            if (this == &other) return;
24            size = other.size;
25            if (data) delete[] data;
26            data = new char[size];
27            std::strcpy(data, other.data);
28            return *this;
29        }
30        String& operator = (String&& other) {
31            size = other.size;
32            data = other.data;
33            other.data = nullptr;
34            other.size = 0;
35            return *this;
36        }
37        ~String() {
38            size = 0;
39            delete[] data;
40            data = nullptr;
41        }
42 };
43
```

`std::strcpy(char* dest, const char* src)` – функция, определённая в заголовочном файле `<cstring>`. Копирует строку символов, на которую указывает `src`, включая нулевой завершающий символ, в массив символов, на первый элемент которого указывает `dest`. Массив `dest` должен не перекрывать `src` и быть достаточного размера.

Сравните реализацию конструкторов копирования и перемещения, операторов присвоения копирования и перемещения.

Дорогими являются не только операции копирования данных, но и выделения и освобождения памяти. Поэтому уменьшение количества таких операций в программе ускоряет её работу.

До C++11 в классах были только конструктор копирования, оператор присвоения копирования и деструктор. Начиная с C++11 к ним добавились конструктор перемещения и оператор присвоения перемещени. Они должны быть реализованы разработчиком при наличие полей с динамической памятью. Если таких полей нет, то компилятор сам правильно их реализует.

LVALUE И RVALUE

lvalue ссылка – это ссылка на не временный объект. У этого объекта можно взять адрес и можно его изменить.

В коде её можно легко отличить от rvalue тем, что данный объект в текущем контексте связан с переменной, т.е. имеет имя, которое в дальнейшем где-то используется.

Например:

```
std::string str = "Текст"; // str – lvalue
```

```
std::string get_text(); // Возвращаемое из функции  
значение – rvalue
```

```
std::string str = get_text(); // справа стоит rvalue  
и поэтому для инициализации lvalue str будет вызван  
конструктор перемещения.
```



Возвращать ссылку на временный объект нельзя.

Она станет «битой» и это приведёт к UB.

Т.е. для функции `std::string get_text();` следующий код неправильный:

```
std::string& str = get_text();
```

Если надо продлить время жизни временного объекта, то можно принять его по константной lvalue или rvalue ссылке:

```
const std::string& str = get_text();
```

или

```
std::string&& str = get_text();
```

будут работать корректно.

ИТОГИ



Благодаря Move семантике стало возможным возвращать большие объекты из функций по значению, не теряя при этом в производительности.